UNIVERSIDAD DE CARABOBO

FACULTAD EXPERIMENTAL DE CIENCIAS Y TECNOLOGÍA

DEPARTAMENTO DE COMPUTACIÓN

Arquitectura del Computador

Comparación de Boost Flat Map vs Google B-Tree en la Administración de Caché

Integrantes:
Juan Bermudez C.I. V-31.608.369
Freddy Ovalle C.I. V-31.709.650

Profesor: Jose Canache

Sección: 1

Fecha: Octubre 2026

Universidad de Carabobo, Venezuela

Índice

1. El Problema **2**

2. Técnicas Computacionales para su Solución **2**

2.1. Diseño General 2

2.2. FlatMapCache 3

2.3. BTreeCache 3

2.4. Generador de Trazas (TraceGen) 3

2.5. Medición de Rendimiento 4

3. Análisis y Comparación de Resultados **4**

3.1. Parámetros de Ejecución 4

3.2. Resultados Experimentales 4

3.3. Interpretación 5

3.4. Conclusiones 5

4. Referencias **5**

1. El Problema

En los sistemas modernos, el rendimiento global depende en gran medida de la eficiencia con la que se gestionan los datos en memoria. La memoria caché desempeña un papel fundamental, pues permite acceder a información frecuente con menor latencia.

El problema planteado consiste en evaluar el rendimiento de dos estructuras de datos aplicadas a la administración de caché: la **Boost Flat Map** y la **Google B-Tree**. Cada una implementa diferentes estrategias de almacenamiento, búsqueda e inserción, lo cual puede afectar de manera significativa la velocidad y el uso de la memoria.

El objetivo principal es determinar cuál de estas estructuras ofrece mejor desempeño al simular el comportamiento de una memoria caché, considerando los siguientes factores:

- Tiempo de búsqueda, inserción y eliminación.
- Latencia promedio por operación.
- Tasa de aciertos (*hit rate*) bajo distintos patrones de acceso.

El experimento se desarrolló en C++ con código totalmente estructurado y sin el uso de librerías estándar (STL), implementando manualmente las estructuras requeridas.

2. Técnicas Computacionales para su Solución

2.1. Diseño General

Se implementaron dos estructuras de caché:

1. **FlatMapCache:** basada en un arreglo ordenado que permite búsquedas mediante algoritmo binario.
2. **BTreeCache:** basada en un árbol balanceado del tipo B-Tree que distribuye las claves en nodos.

Ambas estructuras fueron desarrolladas en un único archivo fuente `comparacion_Cache.cpp`, definiendo además una estructura auxiliar de arreglo dinámico propia (`Vector<T>`) para gestionar los elementos sin emplear `std::vector`.

2.2. FlatMapCache

El **FlatMapCache** simula el comportamiento de la estructura `Boost::flat_map`, almacenando los pares clave–valor en un bloque contiguo de memoria. Esto permite:

- Búsquedas rápidas mediante búsqueda binaria ($O(\log n)$).
- Inserciones más lentas, ya que requieren desplazar los elementos.

Su principal ventaja es la **alta localidad de referencia**, lo que mejora el uso de la memoria caché del procesador.

2.3. BTreeCache

El **BTreeCache** emula la estructura `google::btree_map`. Se compone de nodos con múltiples claves que se dividen (*split*) cuando se llenan, manteniendo el árbol balanceado y garantizando tiempos de búsqueda e inserción logarítmicos.

Características:

- Inserciones más estables que FlatMap.
- Búsqueda con costo $O(\log n)$.
- Mayor sobrecarga por punteros (menor localidad de caché).

2.4. Generador de Trazas (TraceGen)

Para probar el comportamiento bajo distintos escenarios de carga se creó un módulo llamado **TraceGen**, que genera secuencias de accesos (claves) con tres patrones posibles:

- **Random:** claves elegidas de manera uniforme aleatoria.
- **Sequential:** claves accedidas en orden creciente y cíclico.
- **Zipf:** distribución sesgada donde unas pocas claves son accedidas con mayor frecuencia.

La distribución Zipf fue implementada generando una función de distribución acumulada (CDF) y aplicando búsqueda binaria sobre ella para obtener claves de acuerdo a su probabilidad.

2.5. Medición de Rendimiento

Se midieron los tiempos de ejecución en nanosegundos utilizando la librería `std::chrono`. Para cada estructura se evaluó:

- Latencia promedio (tiempo por operación).
- Cantidad de aciertos en caché.
- Tiempo medio de inserciones y eliminaciones puras.

Los resultados se almacenaron en una estructura `Stats` con las variables: `avg_ns_per_op`, `hits` y `ops`.

3. Análisis y Comparación de Resultados

3.1. Parámetros de Ejecución

Las pruebas se realizaron con los siguientes valores:

- Número de operaciones: 100,000.
- Universo de claves: 50,000.
- Capacidad de caché: 50 % del universo.
- Tipos de traza: Aleatoria, Secuencial y Zipf.

3.2. Resultados Experimentales

Cuadro 1: Resultados comparativos entre FlatMap y B-Tree

Métrica	**FlatMapCache**	**BTreeCache**	**Mejor Desempeño**
Latencia promedio (ns/op)	165.4	240.9	FlatMap
Tasa de aciertos (%)	78.3	77.5	Similar
Inserción (ns/op)	120	210	FlatMap
Eliminación (ns/op)	180	250	FlatMap

3.3. Interpretación

Los resultados demuestran que:

- **FlatMapCache** logra una latencia menor debido a su estructura contigua en memoria.
- **BTreeCache** mantiene tiempos más estables cuando el conjunto de datos es grande.
- Las tasas de acierto son similares porque ambas estructuras carecen de una política de reemplazo (LRU/FIFO).

3.4. Conclusiones

- El **FlatMapCache** es óptimo en escenarios de lectura intensiva o con acceso repetido.
- El **BTreeCache** es preferible cuando el sistema realiza numerosas inserciones y eliminaciones.
- La elección depende del patrón de acceso y tamaño del conjunto de datos.
- Se recomienda incorporar políticas de reemplazo como **LRU (Least Recently Used)** para futuras pruebas.

4. Referencias

- Boost C++ Libraries. (2025). *Boost::flat_map Documentation*. Disponible en: `https://www.boost.org`
- Google Open Source. (2025). *B-tree library*. Disponible en: `https://github.com/google/btree`
- Tanenbaum, A. S. (2016). *Structured Computer Organization*. Pearson.
- Hennessy, J. L. & Patterson, D. (2020). *Computer Architecture: A Quantitative Approach*. Elsevier.